



UNL

Universidad
Nacional
de Loja

UNIVERSIDAD NACIONAL DE LOJA
**FACULTAD DE LA ENERGÍA, LAS INDUSTRIAS
Y LOS RECURSOS NATURALES NO RENOVABLES**

**CARRERA DE
TECNOLOGÍAS DE LA INFORMACIÓN**

**Trabajo Autónomo:
Ensayo y Aplicaciones de
Estructuras de Árboles**

ASIGNATURA: Matemáticas Discretas

DOCENTE: Ing. Mario Enrique Cueva Hurtado

ESTUDIANTES:

- José Camilo Merino Morocho
- Edgar Fabricio Ríos Cuenca
- Ashley Patricia Mocha Valarezo
- Abraham Daniel Maza Lapo

CICLO: II Ciclo

FECHA DE ENTREGA: 15 de julio de 2026

**LOJA – ECUADOR
2026**



INTRODUCCIÓN

Los árboles constituyen una de las estructuras matemáticas más importantes dentro de la teoría de grafos y las ciencias de la computación. Se consideran un caso particular de los grafos, caracterizados por ser conexos y no contener ciclos, lo que permite organizar información de manera jerárquica y eficiente. Gracias a estas propiedades, los árboles se han convertido en la base de numerosas aplicaciones informáticas, desde los sistemas de archivos de un computador hasta las bases de datos, compiladores, motores de búsqueda y algoritmos de inteligencia artificial.

A diferencia de un grafo general, un árbol establece una única ruta entre cualquier par de vértices, evitando redundancias y facilitando la optimización de procesos de búsqueda y organización de información. Esta característica permite desarrollar algoritmos con menor complejidad computacional y mayor eficiencia, razón por la cual los árboles son ampliamente utilizados en ingeniería informática, redes de comunicación y desarrollo de software.

Dentro de esta estructura existen diversas clasificaciones, como los árboles binarios, completos, balanceados y enraizados, cada uno diseñado para resolver problemas específicos. Asimismo, los árboles sirven como fundamento para técnicas avanzadas de compresión de datos, como los códigos de Huffman, y para la implementación de árboles de búsqueda binaria que permiten localizar información rápidamente.

El objetivo del presente ensayo es analizar los fundamentos teóricos de los árboles, estudiar sus principales tipos, comprender el funcionamiento de los árboles enraizados y los códigos de prefijos, además de examinar el comportamiento de los árboles de búsqueda binaria y los recorridos fundamentales que permiten procesar la información almacenada en estas estructuras.

DESARROLLO

Parte A. Fundamentos Teóricos y Tipos de Árboles

Definición de árbol

En la materia de Matemáticas Discretas, un árbol se define como un grafo conexo que no contiene ciclos. Esto significa que todos sus vértices están conectados entre sí mediante un único camino posible, evitando conexiones redundantes que puedan formar circuitos.

Desde el punto de vista matemático, un árbol con n vértices siempre posee $n-1$ aristas, propiedad que permite identificar rápidamente si un grafo puede considerarse un árbol.

Las principales características de un árbol son:

- Es conexo.
- No posee ciclos.
- Existe un único camino entre dos vértices.
- Si se elimina una arista, el árbol deja de estar conectado.
- Si se agrega una nueva arista, automáticamente aparece un ciclo.

Estas propiedades convierten a los árboles en estructuras ideales para representar relaciones jerárquicas y optimizar procesos computacionales.

Tipos de árboles

Dependiendo de su estructura y organización, los árboles pueden clasificarse en diferentes categorías.

1. Árbol libre

No posee un nodo raíz definido y cualquiera de sus vértices puede considerarse como punto de referencia.

2. Árbol enraizado

Posee un nodo principal denominado raíz, desde el cual se organizan todos los demás nodos formando distintos niveles jerárquicos.

3. Árbol binario

Cada nodo puede tener como máximo dos hijos, conocidos como hijo izquierdo e hijo derecho. Es uno de los árboles más utilizados en programación.

4. Árbol binario completo

Todos los niveles se encuentran completamente llenos, excepto posiblemente el último, cuyos nodos se ubican de izquierda a derecha.

5. Árbol binario perfecto

Todos los niveles están completamente llenos y todas las hojas se encuentran en el mismo nivel.

6. Árbol balanceado

La diferencia de altura entre los subárboles izquierdo y derecho es mínima, permitiendo realizar búsquedas mucho más rápidas.

7. Árbol AVL

Es un árbol binario balanceado automáticamente mediante rotaciones cada vez que se inserta o elimina un nodo.

8. Árbol B y B+

Se emplean principalmente en sistemas gestores de bases de datos y sistemas de archivos debido a que permiten almacenar grandes cantidades de información con muy pocas operaciones de acceso.

Aplicaciones de los árboles

- Los árboles tienen aplicaciones prácticamente en todas las áreas de la informática.
- En sistemas operativos permiten organizar directorios y carpetas mediante estructuras jerárquicas.
- En compiladores representan expresiones matemáticas y sintácticas mediante árboles de análisis.
- En inteligencia artificial permiten construir árboles de decisión para clasificación y aprendizaje automático.
- También son utilizados en videojuegos, motores de búsqueda, sistemas GPS, análisis genético, procesamiento de imágenes y compresión de archivos.

Parte B. Árboles Enraizados y Optimización

Árboles enraizados

Un árbol enraizado es aquel donde existe un nodo especial denominado raíz, a partir del cual se organizan todos los demás nodos.

Dentro de esta estructura aparecen conceptos fundamentales como:

- Raíz.
- Padre.
- Hijo.
- Hermanos.
- Hoja.
- Nodo interno.
- Nivel.
- Altura.
- Profundidad.

Gracias a esta organización es posible representar estructuras jerárquicas como organigramas, árboles genealógicos, menús de aplicaciones y sistemas de archivos.

Longitud de paseo en árboles

La longitud de un paseo corresponde al número de aristas recorridas para desplazarse desde un nodo inicial hasta un nodo final.

Debido a que en un árbol únicamente existe un camino posible entre dos vértices, el cálculo de la longitud del paseo resulta sencillo y garantiza siempre la ruta mínima.

Esta propiedad es utilizada en algoritmos de navegación, transmisión de datos y optimización de recorridos.

Código de Prefijos (Código Huffman)

Los códigos de prefijos representan una técnica de compresión de datos basada en árboles binarios.

Su principio consiste en asignar códigos binarios más cortos a los símbolos que aparecen con mayor frecuencia y códigos más largos a aquellos menos frecuentes.

La característica fundamental de un código de prefijos es que ningún código constituye el prefijo de otro, eliminando cualquier posibilidad de ambigüedad durante la decodificación.

El algoritmo de Huffman construye un árbol binario donde los nodos con menor frecuencia se combinan progresivamente hasta obtener un único árbol.

Entre sus principales aplicaciones se encuentran:

- Compresión ZIP.
- Formato JPEG.
- Formato PNG.
- Archivos MP3.
- Compresión de texto.
- Transmisión eficiente de información.

Árboles de búsqueda binaria

Los árboles de búsqueda binaria (ABB) son estructuras ordenadas que permiten localizar información rápidamente.

Su propiedad principal establece que:

- Todos los valores menores se almacenan en el subárbol izquierdo.
- Todos los valores mayores se almacenan en el subárbol derecho.

Gracias a esta organización, operaciones como búsqueda, inserción y eliminación pueden ejecutarse con una complejidad cercana a $O(\log n)$ cuando el árbol permanece balanceado.

Los árboles de búsqueda binaria constituyen la base de numerosos algoritmos empleados en software moderno, bases de datos, compiladores y sistemas de almacenamiento.

Parte C. Recorridos en un Árbol

Introducción a los Recorridos de Árboles

Un recorrido se logra siempre que cada nodo del árbol sea visitado una y solo una vez. En los árboles binarios, el recorrido en profundidad es el más común y se divide en tres tipos, dependiendo del orden en que se visite la raíz respecto a sus subárboles izquierdo (I) y derecho (D).

Tomando como base la metodología de la página 257 y siguientes del texto guía:

Preorden (NID - Nodo, Izquierdo, Derecho):

1. Se visita el nodo raíz (N).
2. Se recorre el subárbol izquierdo (I) en preorden.
3. Se recorre el subárbol derecho (D) en preorden. Lógica: Es útil para copiar la estructura de un árbol o evaluar expresiones en notación prefija.

Inorden/Enorden (IND - Izquierdo, Nodo, Derecho):

1. Se recorre el subárbol izquierdo (I) en inorden.
2. Se visita el nodo raíz (N).
3. Se recorre el subárbol derecho (D) en inorden. Lógica: En un árbol de búsqueda binaria, este recorrido devuelve los valores en orden ascendente.

Postorden (IDN - Izquierdo, Derecho, Nodo):

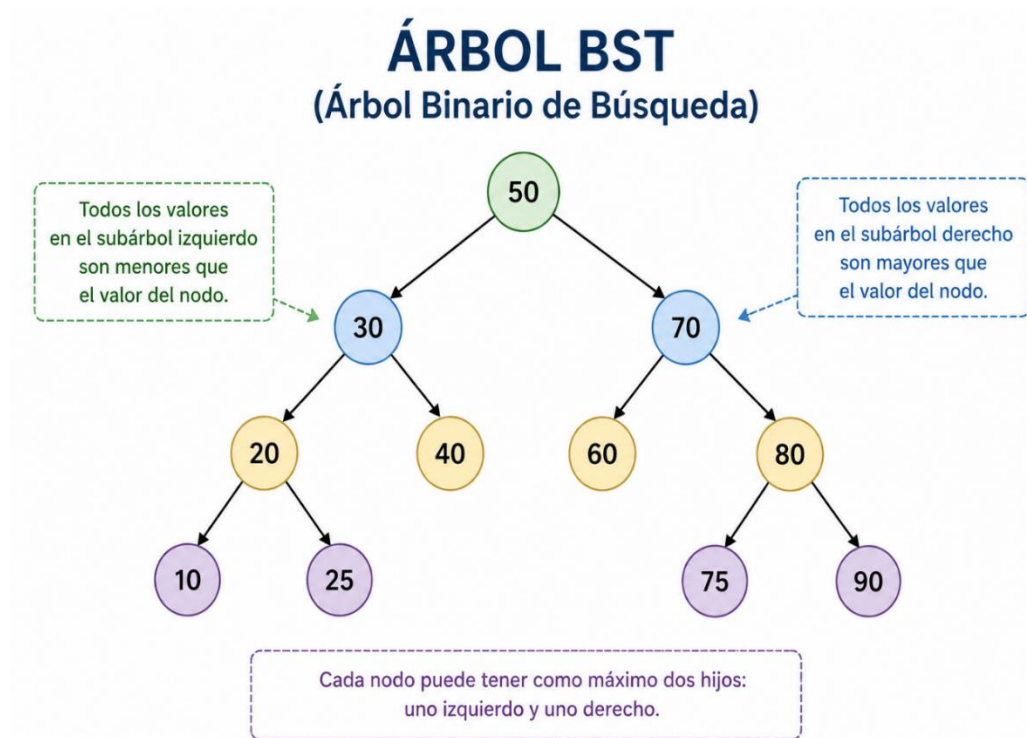
1. Se recorre el subárbol izquierdo (I) en postorden.
2. Se recorre el subárbol derecho (D) en postorden.
3. Se visita el nodo raíz (N). Lógica: Se utiliza comúnmente para eliminar nodos del árbol o evaluar expresiones en notación postfija.

Resolución de Ejercicios Prácticos (4 Niveles de Altura)

1. Los estudiantes construyen árboles binarios de búsqueda (BST) paso a paso, insertando valores numéricos y realizando los tres tipos de recorridos, documentando el orden de visita de nodos en cada caso.

¿Qué es un árbol binario de búsqueda (BST)?

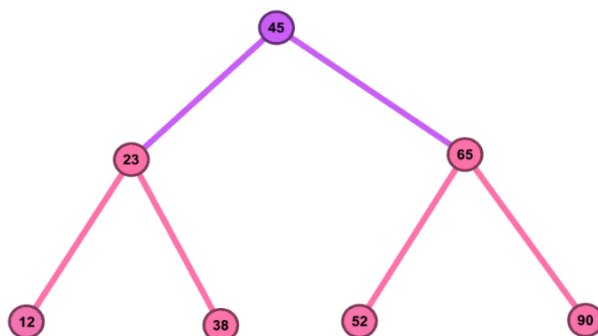
Un Árbol Binario de Búsqueda (BST) es una estructura de datos jerárquica compuesta por nodos, donde cada nodo puede tener como máximo dos hijos y se mantiene la propiedad de que los valores menores se ubican a la izquierda del nodo y los mayores a la derecha, permitiendo realizar búsquedas y operaciones de manera eficiente.



Ejercicio 1:

Planteamiento del problema:

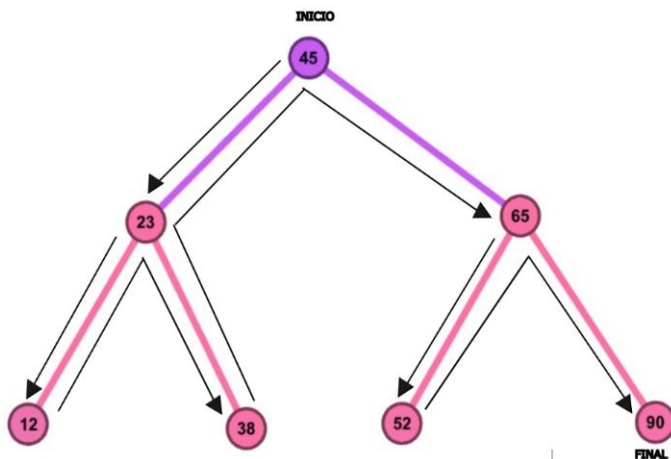
Una tienda de tecnología quiere organizar los códigos de sus productos para optimizar las búsquedas en su base de datos. El sistema registra los códigos según van llegando los camiones de inventario. Se requiere construir el árbol binario de búsqueda para la siguiente secuencia de códigos de productos: [45, 23, 65, 12, 38, 52, 90] y documentar sus tres recorridos para auditoría del sistema.



Preorden (Raíz --- Izquierda --- Derecha):

Anotamos la raíz actual y luego bajamos.

- Anotamos la raíz principal: **45**.
- Vamos al subárbol izquierdo: anotamos **23**, bajamos a su izquierda **12**, subimos e irnos a su derecha **38**.
- Vamos al subárbol derecho: anotamos **65**, bajamos a su izquierda **52**, subimos e irnos a su derecha **90**.

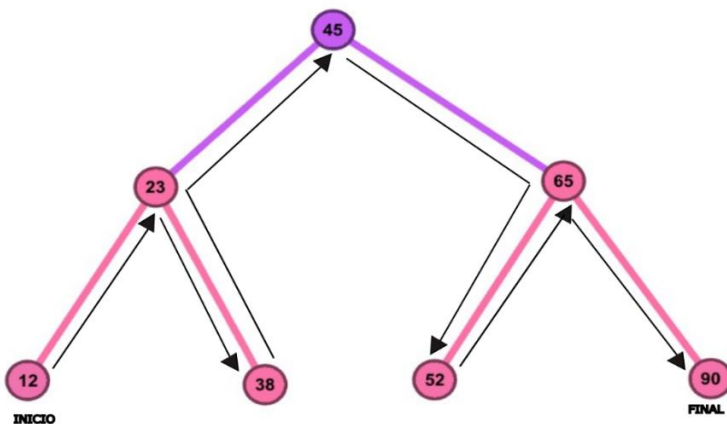


- **Resultado:** [45, 23, 12, 38, 65, 52, 90]

Inorden (Izquierda --- Raíz --- Derecha):

Avanzamos hasta el extremo izquierdo antes de anotar.

- El más a la izquierda es **12**, subimos a su padre **23**, bajamos a su derecha **38**.
- Subimos a la raíz principal **45**.
- Vamos al lado derecho: buscamos el más a la izquierda que es **52**, subimos a su padre **65**, y finalizamos en su derecha **90**.

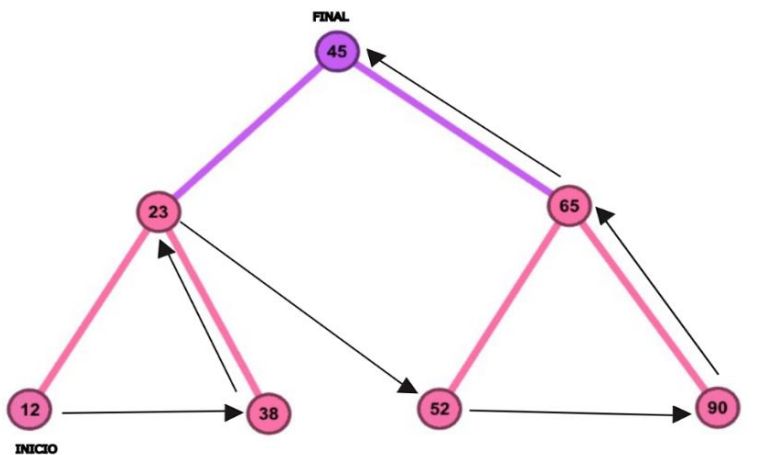


- **Resultado:** [12, 23, 38, 45, 52, 65, 90]

Postorden (Izquierda --- Derecha --- Raíz):

Los padres se procesan al final.

- Hijos del 23: anotamos **12** y **38**. Ahora anotamos al padre: **23**.
- Hijos del 65: anotamos **52** y **90**. Ahora anotamos al padre: **65**.
- Finalmente, anotamos la raíz del árbol completo: **45**.

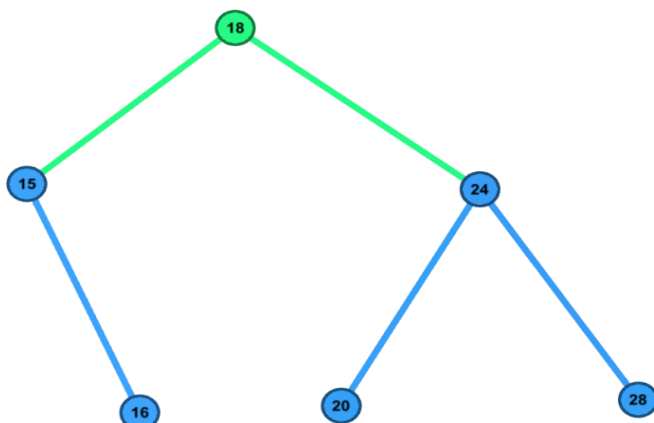


- **Resultado:** [12, 38, 23, 52, 90, 65, 45]

Ejercicio 2:

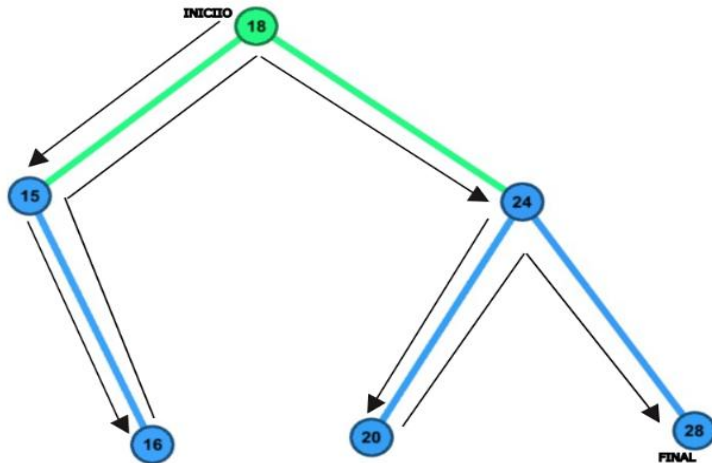
Planteamiento del problema:

Una estación meteorológica registra las temperaturas máximas del mes para evaluar las tendencias de cambio climático. Los sensores envían los datos de manera asíncrona. Se requiere construir un BST para almacenar las siguientes lecturas de temperatura en grados Celsius: [18, 24, 15, 20, 28, 16] y calcular los recorridos para que el software genere los reportes visuales.



Preorden:

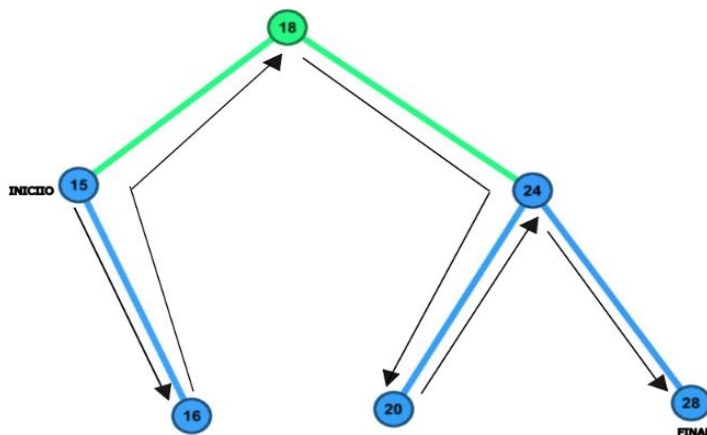
- Raíz principal: **18**.
- Subárbol izquierdo: padre **15**, luego su hijo derecho **16**.
- Subárbol derecho: padre **24**, luego su hijo izquierdo **20**, y su hijo derecho **28**.



- **Resultado:** [18, 15, 16, 24, 20, 28]

Inorden:

- Primero iniciamos en la raíz padre **15**, luego a su hijo derecho **16**.
- Raíz del árbol: **18**.
- Subárbol derecho: el más a la izquierda es **20**, subimos a su padre **24**, terminamos en su derecha **28**.

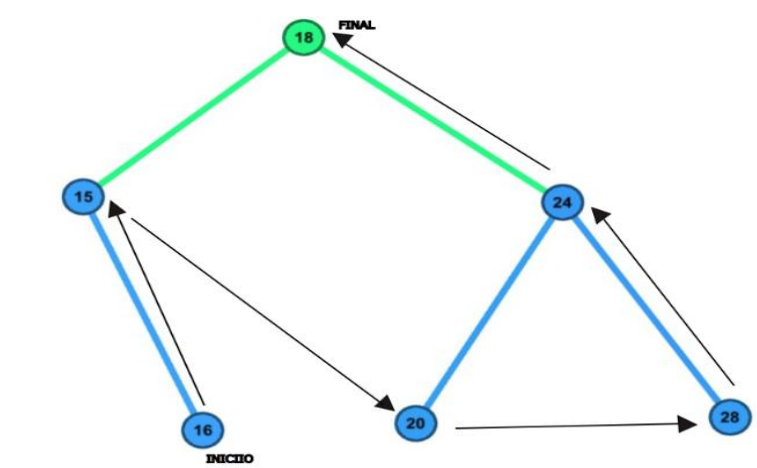


- **Resultado:** [15, 16, 18, 20, 24, 28]

Postorden:

- Subárbol izquierdo: el hijo **16** va antes que su padre **15**.
- Subárbol derecho: procesamos los hijos **20** y **28** antes que a su padre **24**.

- Finalmente, la raíz principal: **18**.



- **Resultado:** [16, 15, 20, 28, 24, 18]

Ejercicio 3:

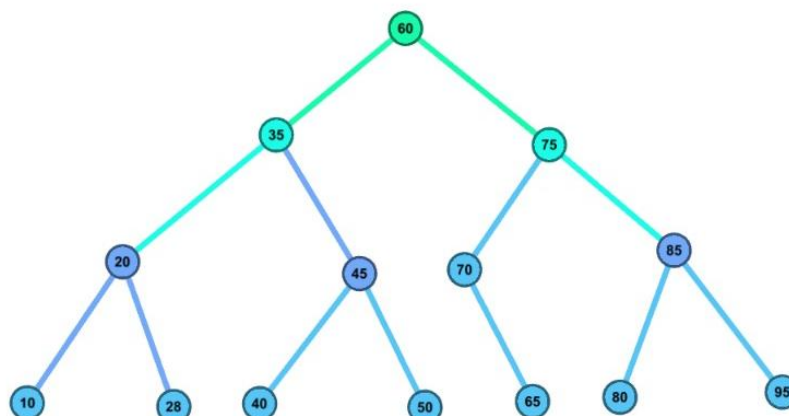
Planteamiento del Problema:

Sistema de Triage en Emergencias Médicas

Un hospital de alta complejidad implementa un sistema automatizado para priorizar la atención en la sala de emergencias. A cada paciente se le asigna un **índice de gravedad numérica** del 1 al 100 mediante un algoritmo médico en el momento en que ingresa (donde los valores representan la criticidad y compatibilidad sintomática).

Para almacenar estos índices de manera organizada y permitir consultas instantáneas, el sistema utiliza un Árbol Binario de Búsqueda (BST). Los pacientes ingresan en el siguiente orden estricto de índices:

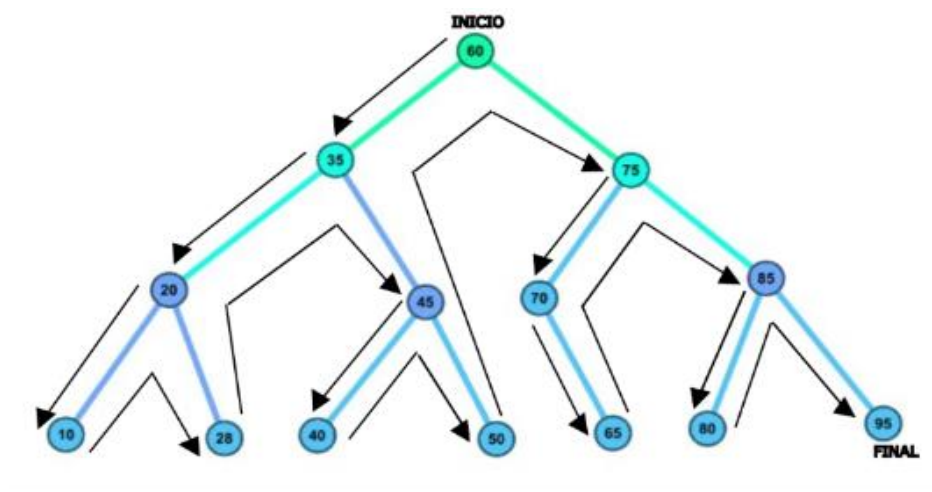
[60, 35, 75, 20, 45, 70, 85, 10, 28, 40, 50, 65, 80, 95]



PREORDEN (Raíz → Izquierda → Derecha)

Se anota el número al tocarlo por primera vez.

1. Raíz principal: **60**
2. Bloque izquierdo del 35: **35 --- 20 --- 10 --- 28**
3. Bloque derecho del 35: **45 --- 40 --- 50**
4. Bloque izquierdo del 75: **75 --- 70 --- 65**
5. Bloque derecho del 75: **85 --- 80 --- 95**

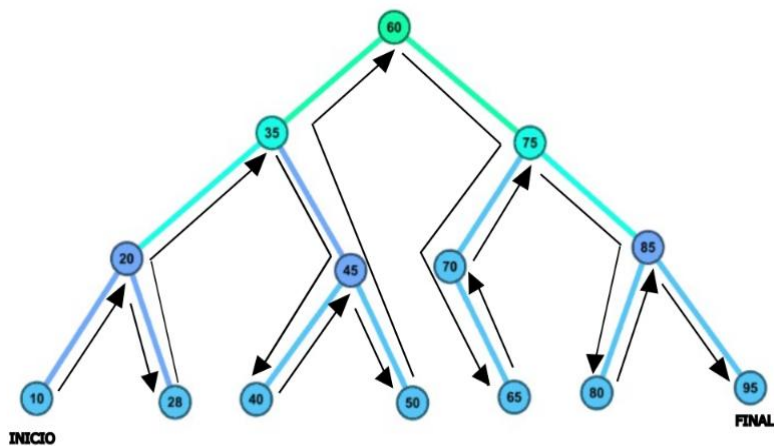


- **Resultado:** [60, 35, 20, 10, 28, 45, 40, 50, 75, 70, 65, 85, 80, 95]

INORDEN (Izquierda → Raíz → Derecha)

Se anota el número al pasar por debajo de él (de menor a mayor).

1. Extremo izquierdo (rama 20): **10 --- 20 --- 28**
2. Sube a la subraíz: **35**
3. Siguiendo rama (rama 45): **40 --- 45 --- 50**
4. Sube a la raíz principal: **60**
5. Siguiendo rama (rama 70): **65 --- 70**
6. Sube a la subraíz: **75**
7. Extremo derecho (rama 85): **80 --- 85 --- 95**

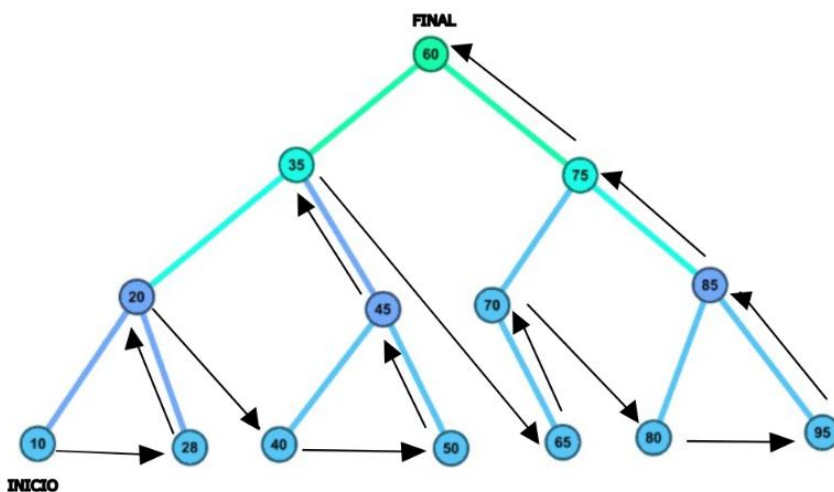


- **Resultado:** [10, 20, 28, 35, 40, 45, 50, 60, 65, 70, 75, 80, 85, 95]

POSTORDEN (Izquierda → Derecha → Raíz)

Se anotan primero los hijos y al final sus padres.

1. Hijos y padre de la rama 20: **10 --- 28 --- 20**
2. Hijos y padre de la rama 45: **40 --- 50 --- 45**
3. Sube al padre de ambos bloques: **35**
4. Hijo y padre de la rama 70: **65 --- 70**
5. Hijos y padre de la rama 85: **80 --- 95 --- 85**
6. Sube al padre de ambos bloques: **75**
7. Al final de todo, la raíz principal: **60**



- **Resultado:** [10, 28, 20, 40, 50, 45, 35, 65, 70, 80, 95, 85, 75, 60]

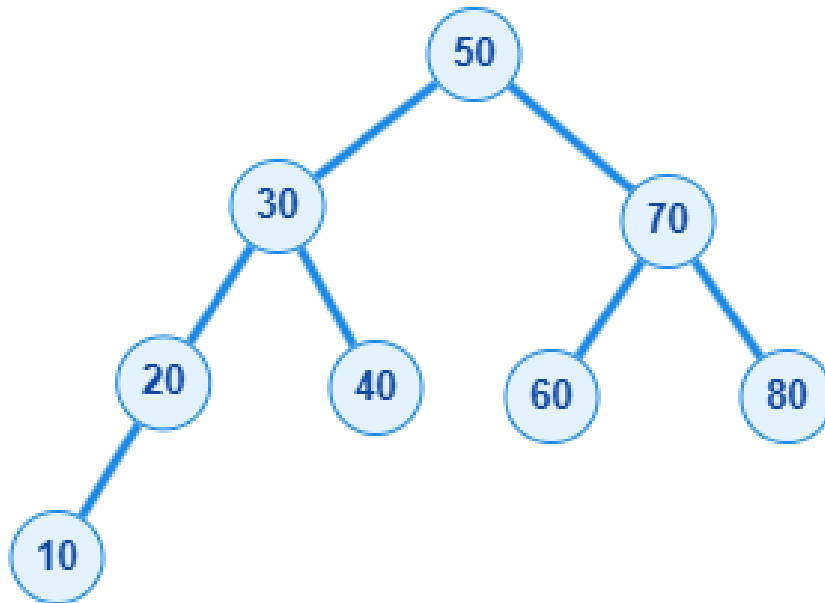
Ejercicio 4

Claves: {50, 30, 70, 20, 40, 60, 80, 10}

Estructura por niveles:

- Nivel 1: 50
- Nivel 2: 30, 70
- Nivel 3: 20, 40, 60, 80
- Nivel 4: 10 (Izquierdo de 20)

Representación Gráfica:



Recorrido	Lógica de Resolución	Resultado Final	Imagen
Preorden	Raíz → Izquierdo → Derecho. Permite duplicar u obtener una estructura idéntica del árbol original.	50, 30, 20, 10, 40, 70, 60, 80	
Inorden	En un árbol de búsqueda binaria, el recorrido inorden siempre da como resultado los elementos ordenados ascendentemente.	10, 20, 30, 40, 50, 60, 70, 80	

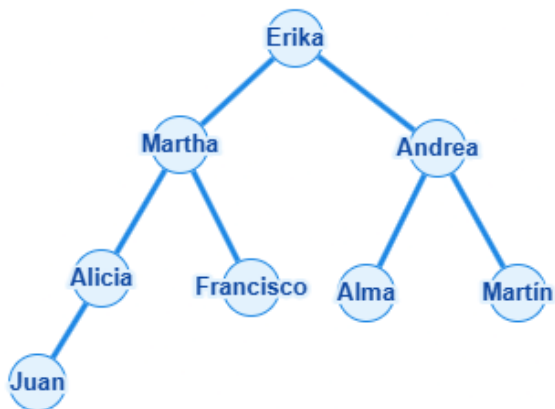
Recorrido	Lógica de Resolución	Resultado Final	Imagen
Postorden	Hijos izquierdos y derechos procesados por completo antes de su respectivo nodo padre.	10, 20, 40, 30, 60, 80, 70, 50	

Ejercicio 5:

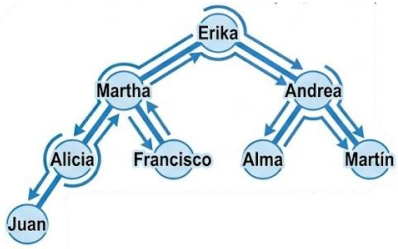
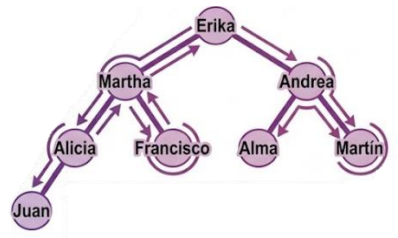
Estructura por niveles:

- Nivel 1: Erika
- Nivel 2: Martha (Izquierda), Andrea (Derecha)
- Nivel 3: Alicia (Izquierda de Martha), Francisco (Derecha de Martha), Alma (Izquierda de Andrea), Martín (Derecha de Andrea)
- Nivel 4: Juan (Izquierda de Alicia)

Representación Gráfica:



Recorrido	Lógica de Resolución	Resultado Final	Imagen
Preorden	Erika (Raíz) → Subárbol Martha (Martha, Alicia, Juan, Francisco) →	Erika, Martha, Alicia, Juan, Francisco,	

Recorrido	Lógica de Resolución	Resultado Final	Imagen
	Subárbol Andrea (Andrea, Alma, Martín).	Andrea, Alma, Martín	
Inorden	Recorrido desde Juan → Alicia → Martha → Francisco (rama izquierda completa) → Erika (Raíz central) → Alma → Andrea → Martín.	Juan, Alicia, Martha, Francisco, Erika, Alma, Andrea, Martín	
Postorden	Se completan las hojas más profundas primero: Juan → Alicia → Francisco → Martha, luego Alma → Martín → Andrea, finalizando en Erika.	Juan, Alicia, Francisco, Martha, Alma, Martín, Andrea, Erika	

2. Los estudiantes investigan de forma autónoma sobre la relación entre la teoría de grafos/árboles y las estructuras de datos utilizadas en programación (heaps, tries, árboles B), bases de datos (índices) e inteligencia artificial (árboles de decisión). Elaboran un miniinforme con exposición apoyada en recursos TIC.

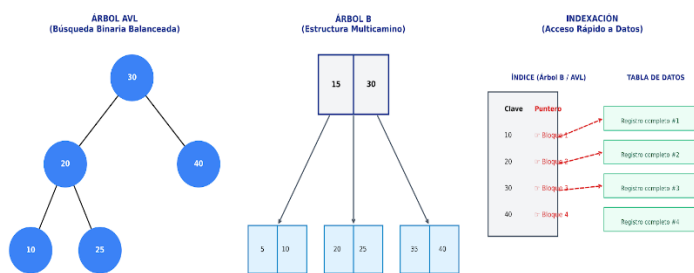
Las estructuras de datos se consideran los bloques de construcción fundamentales de la informática moderna, permitiendo el almacenamiento y la manipulación eficiente de la información necesaria para diseñar algoritmos complejos. Estas estructuras no solo organizan los datos, sino que son esenciales para el rendimiento de aplicaciones en diversos dominios técnicos.

1. Fundamentos Teóricos: Árboles y Grafos

3. Gestión de Información y Bases de Datos (Árboles B e Índices)

En el ámbito del almacenamiento masivo, las estructuras deben minimizar los tiempos de acceso a memoria secundaria:

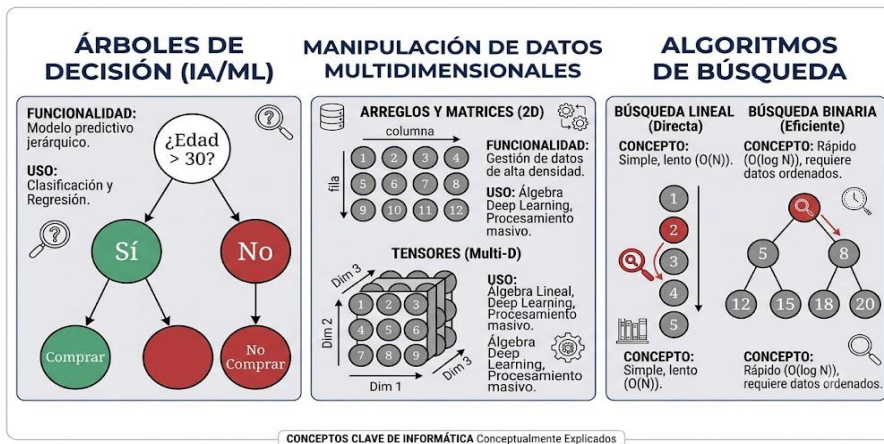
- **Árbol B:** Este tipo de árbol auto balanceado está diseñado específicamente para manejar grandes cantidades de datos en disco. El texto indica que son el estándar en **sistemas de archivos y bases de datos** donde la lectura y escritura eficientes desde el disco son críticas.
- **Árboles AVL:** Son árboles binarios de búsqueda que mantienen una diferencia mínima de altura entre sus subárboles para garantizar tiempos de búsqueda rápidos (). Se utilizan comúnmente en la **indexación de bases de datos** y en tablas de símbolos.
- **Indexación:** Se señala que las bases de datos dependen de árboles B, tablas hash y diversas formas de indexación para permitir la recuperación de registros específicos entre millones de datos de forma precisa y rápida.



4. Inteligencia Artificial y Machine Learning

El campo de la IA utiliza estas estructuras tanto para el modelado lógico como para la gestión de datos a gran escala:

- **Árboles de Decisión:** La estructura jerárquica de los árboles es la base de los algoritmos de árboles de decisión, utilizados para clasificar información y facilitar la toma de decisiones automatizada.
- **Manipulación de Datos Multidimensionales:** En el aprendizaje automático, se emplean estructuras como **arreglos, matrices y tensores** para almacenar y procesar datos masivos de múltiples dimensiones.
- **Algoritmos de Búsqueda:** La IA también se apoya en árboles binarios y tablas hash para implementar procesos eficientes de recuperación de información.



5. Optimización, Compromisos y Futuro

La elección de una estructura sobre otra implica considerar **trade-offs** o compromisos técnicos. El desarrollador debe evaluar la **complejidad temporal** (tiempo de ejecución) y la **complejidad espacial** (memoria requerida). Por ejemplo, mientras que las tablas hash ofrecen búsquedas casi instantáneas (), su mantenimiento suele ser más difícil en comparación con los arreglos simples.

Hacia el futuro, la investigación se está centrando en el desarrollo de estructuras de datos adaptadas a:

- **Big Data:** Manejo de datasets masivos en entornos distribuidos.
- **Blockchain:** Estructuras que soporten la descentralización, inmutabilidad y el consenso.
- **Computación Cuántica:** Optimización de estructuras para nuevos entornos de procesamiento.
- **Seguridad y Privacidad:** Creación de formas de almacenamiento que garanticen la protección de datos sensibles.

Portafolio de matemáticas discreta: con todos los trabajos realizados; distribuidos en ACD, APE, AA, AB

José Camilo Merino Morocho

<https://github.com/JoseCamilo667/portafolioDigitalMD/blob/main/Index..md>

Edgar Fabricio Ríos Cuenca

https://github.com/riosfabricio444-ai/Portafolio_Matematicas_Discretas/blob/main/README.md

Ashley Patricia Mocha Valarezo

<https://github.com/ashm-crypto/PortafolioMD/blob/main/Portafolio.md>

Abraham Daniel Maza Lapo

<https://github.com/abrahammaza2411-cmd/PortafolioU3/blob/main/Portafolio.md>

CONCLUSIONES

- Los árboles representan una de las estructuras más importantes de las Matemáticas Discretas debido a que permiten organizar información de manera jerárquica sin formar ciclos, garantizando un único camino entre dos nodos.
- Se comprobó que los diferentes tipos de árboles poseen aplicaciones específicas en áreas como bases de datos, sistemas operativos, inteligencia artificial y redes de computadoras, convirtiéndose en herramientas fundamentales para la ingeniería informática.
- Los árboles enraizados, los códigos de Huffman y los árboles de búsqueda binaria demuestran cómo una estructura matemática puede optimizar procesos de almacenamiento, búsqueda y compresión de información, reduciendo significativamente el tiempo de ejecución de los algoritmos.
- Finalmente, los recorridos Preorden, Inorden y Postorden constituyen procedimientos esenciales para acceder y procesar la información almacenada en los árboles, siendo utilizados ampliamente en programación, compiladores y procesamiento de datos.

REFLEXIÓN PERSONAL

José Camilo Merino Morocho

Durante el desarrollo de este trabajo comprendí que los árboles son estructuras fundamentales para organizar y procesar información de manera eficiente. Lo que más me llamó la atención fue el funcionamiento de los árboles de búsqueda binaria y cómo permiten realizar búsquedas rápidas. Como futuro ingeniero en Computación, este conocimiento me ayudará a desarrollar programas más eficientes y comprender mejor el funcionamiento interno de bases de datos y sistemas informáticos.

Edgar Fabricio Ríos Cuenca

Lo más interesante para mí fue conocer los códigos de prefijo, especialmente el algoritmo de Huffman, ya que demuestra cómo las matemáticas permiten optimizar el almacenamiento y la transmisión de información. Considero que estos conocimientos serán de gran utilidad en mi formación profesional, pues comprenderé mejor cómo funcionan los procesos de compresión de archivos y las comunicaciones digitales.

Ashley Patricia Mocha Valarezo

Al estudiar los recorridos de los árboles entendí que un mismo conjunto de datos puede procesarse de distintas maneras según el objetivo que se quiera alcanzar. Aprender los recorridos Preorden, Inorden y Postorden fortaleció mi razonamiento lógico y mi capacidad para analizar algoritmos. En mi futuro profesional podré aplicar estos conceptos en el desarrollo de software y en la creación de estructuras de datos eficientes.

Abraham Daniel Maza Lapo

Este ensayo me permitió comprender que los árboles no son únicamente conceptos matemáticos, sino herramientas ampliamente utilizadas en la informática moderna. Entender sus propiedades y aplicaciones me ayudó a relacionar la teoría con problemas reales de programación. Estoy seguro de que estos conocimientos serán

una base importante para aprender estructuras de datos más avanzadas y desarrollar soluciones informáticas con mayor eficiencia.

REFERENCIAS

1. Villalpando Becerra, J. F., & García Sandoval, A. (2014). *Matemáticas discretas: Aplicaciones y ejercicios*. Grupo Editorial Patria.
2. Epp, S. S. (2012). *Matemáticas discretas con aplicaciones* (4.^a ed.). Cengage Learning.
3. Levin, O. (2025). *Discrete Mathematics: An Open Introduction* (4th ed.). University of Northern Colorado.
4. Srihith, I. V., Donald, A., Srinivas, T. A., Reddy, D. A., & Varaprasad, R. (2023). The backbone of computing: An exploration of data structures. *International Journal of Advanced Research in Science, Communication and Technology*, 155–163. <https://doi.org/10.48175/IJARSCT-9105>